Exemple code IndexedDB

donne moi un exemple de code js qui crée une base indexedBD 'dataJL' qui contient une table 'tbl_JL' avec 3 champs, id=long, nom=string, prénom=string

Voici un **exemple complet en JavaScript** qui :

- Crée une base **IndexedDB** nommée `dataJL`.
- Ajoute un **object store** `tbl_JL` avec trois champs :
  - `id` (**clé primaire**, de type `long` / `auto-increment`)
  - `nom` (**string**)
  - `prenom` (**string**)

Il inclut aussi un exemple d'insertion et de lecture pour tester la base.

Code JavaScript

```js
// Nom et version de la base
const dbName = "dataJL";
const dbVersion = 1;
let db;

// Ouverture/création de la base
const request = indexedDB.open(dbName, dbVersion);

// Gestion de la création ou mise à jour du schéma
request.onupgradeneeded = function (event) {
  db = event.target.result;

  // Crée un object store "tbl_JL" avec "id" en clé primaire auto-incrémentée
  const store = db.createObjectStore("tbl_JL", {
    keyPath: "id",
    autoIncrement: true
  });

  // Index pour les recherches rapides
  store.createIndex("nom", "nom", { unique: false });
  store.createIndex("prenom", "prenom", { unique: false });
```

```
  console.log("Base 'dataJL' et table 'tbl_JL' créées !");
};

// Succès d'ouverture
request.onsuccess = function (event) {
  db = event.target.result;
  console.log("Connexion réussie à la base 'dataJL'");

  // Exemple d'insertion
  addPerson("Dupont", "Jean");
  addPerson("Martin", "Claire");

  // Lecture des données
  getAllPersons();
};

// Gestion d'erreurs
request.onerror = function (event) {
  console.error("Erreur d'ouverture :", event.target.error);
};

// ---- Fonctions utilitaires ----

// Ajout d'un enregistrement
function addPerson(nom, prenom) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  const person = { nom, prenom };
  const addRequest = store.add(person);

  addRequest.onsuccess = () => {
    console.log(`Ajouté : ${nom} ${prenom}`);
  };

  addRequest.onerror = (event) => {
    console.error("Erreur d'ajout :", event.target.error);
  };
}

// Récupération de tous les enregistrements
function getAllPersons() {
```

```
  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  const request = store.getAll();

  request.onsuccess = () => {
   console.log("Contenu de tbl_JL :", request.result);
  };

  request.onerror = (event) => {
   console.error("Erreur de lecture :", event.target.error);
  };
}
```

Comment utiliser ce script

1. Copiez ce code dans un fichier `index.html` dans un bloc `<script>` ou dans un fichier `.js` externe.
2. Ouvrez la page dans un navigateur moderne (Chrome, Firefox, Edge).
3. Ouvrez la **console du navigateur (F12 → Console)** pour voir les messages.
4. Vous verrez la création de la base, l'ajout de deux personnes, et l'affichage du contenu.

Veux-tu que je t'ajoute aussi un exemple de **mise à jour** ou **suppression** d'un enregistrement ?

oui

Voici la **version mise à jour** du script avec :

- **Insertion** de données (`addPerson`)
- **Lecture** de toutes les données (`getAllPersons`)
- **Mise à jour** d'un enregistrement (`updatePerson`)
- **Suppression** d'un enregistrement (`deletePerson`)

Code JavaScript complet

```js
// Nom et version de la base
const dbName = "dataJL";
const dbVersion = 1;
let db;

// Ouverture/création de la base
const request = indexedDB.open(dbName, dbVersion);

// Création de la structure si nouvelle base ou version modifiée
request.onupgradeneeded = function (event) {
  db = event.target.result;

  const store = db.createObjectStore("tbl_JL", {
    keyPath: "id",
    autoIncrement: true
  });

  store.createIndex("nom", "nom", { unique: false });
  store.createIndex("prenom", "prenom", { unique: false });

  console.log("Base 'dataJL' et table 'tbl_JL' créées !");
};

// Succès d'ouverture
request.onsuccess = function (event) {
  db = event.target.result;
  console.log("Connexion réussie à la base 'dataJL'");

  // --- EXEMPLES ---
  addPerson("Dupont", "Jean");
  addPerson("Martin", "Claire");

  setTimeout(() => {
    getAllPersons();

    // Mise à jour du premier enregistrement (id=1)
    updatePerson(1, "Durand", "Jean-Pierre");

    // Suppression du deuxième enregistrement (id=2)
    deletePerson(2);
```

```
    // Lecture après modification
    setTimeout(getAllPersons, 500);
  }, 1000);
};

// Gestion d'erreurs
request.onerror = function (event) {
  console.error("Erreur d'ouverture :", event.target.error);
};

// ======================
//    UTILITAIRES
// ======================

// --- Ajout d'un enregistrement ---
function addPerson(nom, prenom) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  const person = { nom, prenom };
  const addRequest = store.add(person);

  addRequest.onsuccess = () => {
    console.log(`Ajouté : ${nom} ${prenom}`);
  };
  addRequest.onerror = (event) => {
    console.error("Erreur d'ajout :", event.target.error);
  };
}

// --- Lecture de tous les enregistrements ---
function getAllPersons() {
  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  const request = store.getAll();

  request.onsuccess = () => {
    console.log("Contenu de tbl_JL :", request.result);
  };
  request.onerror = (event) => {
    console.error("Erreur de lecture :", event.target.error);
  };
```

```
}

// --- Mise à jour d'un enregistrement par ID ---
function updatePerson(id, nouveauNom, nouveauPrenom) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  const getRequest = store.get(id);

  getRequest.onsuccess = () => {
    const data = getRequest.result;
    if (data) {
      data.nom = nouveauNom;
      data.prenom = nouveauPrenom;
      const updateRequest = store.put(data);
      updateRequest.onsuccess = () => {
        console.log(`Mise à jour ID=${id} en ${nouveauNom} ${nouveauPrenom}`);
      };
      updateRequest.onerror = (event) => {
        console.error("Erreur de mise à jour :", event.target.error);
      };
    } else {
      console.warn(`Aucun enregistrement trouvé avec ID=${id}`);
    }
  };
}

// --- Suppression d'un enregistrement par ID ---
function deletePerson(id) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  const deleteRequest = store.delete(id);

  deleteRequest.onsuccess = () => {
    console.log(`Enregistrement ID=${id} supprimé`);
  };
  deleteRequest.onerror = (event) => {
    console.error("Erreur de suppression :", event.target.error);
  };
}
```

Ce que fait le script

1. Crée la base `dataJL` et la table `tbl_JL`.
2. Ajoute deux personnes :
   - `Dupont Jean`
   - `Martin Claire`
3. Met à jour l'enregistrement `id=1` en `Durand Jean-Pierre`.
4. Supprime l'enregistrement `id=2`.
5. Relit la table pour afficher le contenu final.

Veux-tu que je t'ajoute un **exemple avec une interface HTML** pour saisir et gérer les données ?

oui

Voici un **exemple complet avec une interface HTML simple** :

- Formulaire pour **ajouter** une personne.
- Liste affichant toutes les personnes.
- Boutons pour **modifier** ou **supprimer** un enregistrement.

Code complet : `index.html`

html

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Gestion IndexedDB - dataJL</title>
  <script defer src="app.js"></script>
  <style>
    body {
      font-family: Arial, sans-serif;
      background: #f9f9f9;
      margin: 20px;
    }
    h1 {
```

```
    color: #333;
  }
  form {
    margin-bottom: 20px;
    background: #fff;
    padding: 10px;
    border: 1px solid #ccc;
    border-radius: 8px;
  }
  input {
    margin: 5px;
    padding: 5px;
  }
  button {
    padding: 5px 10px;
    margin: 5px;
    cursor: pointer;
  }
  table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 10px;
    background: #fff;
  }
  table, th, td {
    border: 1px solid #ccc;
  }
  th, td {
    padding: 8px;
    text-align: center;
  }
 </style>
</head>
<body>
 <h1>Gestion de la base IndexedDB - dataJL</h1>

 <form id="formPerson">
  <input type="text" id="nom" placeholder="Nom" required>
  <input type="text" id="prenom" placeholder="Prénom" required>
  <button type="submit">Ajouter</button>
 </form>
```

```
  <table id="tablePersons">
    <thead>
      <tr>
        <th>ID</th>
        <th>Nom</th>
        <th>Prénom</th>
        <th>Actions</th>
      </tr>
    </thead>
    <tbody></tbody>
  </table>
</body>
</html>
```

Code JavaScript : `app.js`

```
js

 const dbName = "dataJL";
const dbVersion = 1;
let db;

// ---- Ouverture de la base ----
const request = indexedDB.open(dbName, dbVersion);

request.onupgradeneeded = function (event) {
  db = event.target.result;

  const store = db.createObjectStore("tbl_JL", {
    keyPath: "id",
    autoIncrement: true
  });

  store.createIndex("nom", "nom", { unique: false });
  store.createIndex("prenom", "prenom", { unique: false });
};

request.onsuccess = function (event) {
  db = event.target.result;
  refreshTable();
};
```

```
request.onerror = function (event) {
  console.error("Erreur d'ouverture :", event.target.error);
};

// ---- Formulaire d'ajout ----
document.getElementById("formPerson").addEventListener("submit", function (e) {
  e.preventDefault();
  const nom = document.getElementById("nom").value.trim();
  const prenom = document.getElementById("prenom").value.trim();
  if (nom && prenom) {
    addPerson(nom, prenom);
    this.reset();
  }
});

// ---- Ajout ----
function addPerson(nom, prenom) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  store.add({ nom, prenom }).onsuccess = () => refreshTable();
}

// ---- Lecture ----
function refreshTable() {
  const tbody = document.querySelector("#tablePersons tbody");
  tbody.innerHTML = "";

  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  store.getAll().onsuccess = (event) => {
    const persons = event.target.result;
    persons.forEach((person) => {
      const tr = document.createElement("tr");
      tr.innerHTML = `
        <td>${person.id}</td>
        <td>${person.nom}</td>
        <td>${person.prenom}</td>
        <td>
          <button onclick="editPerson(${person.id}, '${person.nom}',
'${person.prenom}')">Modifier</button>
          <button onclick="deletePerson(${person.id})">Supprimer</button>
```

```
        </td>
      `;
      tbody.appendChild(tr);
    });
  };
}

// ---- Suppression ----
function deletePerson(id) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  store.delete(id).onsuccess = () => refreshTable();
}

// ---- Modification ----
function editPerson(id, nom, prenom) {
  const nouveauNom = prompt("Nouveau nom :", nom);
  const nouveauPrenom = prompt("Nouveau prénom :", prenom);
  if (nouveauNom && nouveauPrenom) {
    const transaction = db.transaction("tbl_JL", "readwrite");
    const store = transaction.objectStore("tbl_JL");
    store.get(id).onsuccess = (event) => {
      const data = event.target.result;
      data.nom = nouveauNom;
      data.prenom = nouveauPrenom;
      store.put(data).onsuccess = () => refreshTable();
    };
  }
}
```

Comment tester

1. Crée un dossier vide.
2. Ajoute deux fichiers :
   - `index.html` (code HTML ci-dessus)
   - `app.js` (code JavaScript ci-dessus).
3. Ouvre `index.html` **dans Chrome, Firefox ou Edge**.
4. Ajoute des personnes via le formulaire.
5. Utilise les boutons **Modifier** et **Supprimer** pour gérer les enregistrements.

Veux-tu que je te montre **comment exporter ces données vers CSV ou JSON** pour les sauvegarder ?

oui

Voici une **version améliorée** qui ajoute deux nouveaux boutons :

- **Exporter en CSV**
- **Exporter en JSON**

Ces boutons génèrent un fichier téléchargeable contenant toutes les données de la table `tbl_JL`.

HTML mis à jour (`index.html`)

Ajoute les boutons sous le tableau :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
 <meta charset="UTF-8">
 <title>Gestion IndexedDB - dataJL</title>
 <script defer src="app.js"></script>
 <style>
  body {
   font-family: Arial, sans-serif;
   background: #f9f9f9;
   margin: 20px;
  }
  h1 {
   color: #333;
  }
  form {
   margin-bottom: 20px;
   background: #fff;
   padding: 10px;
   border: 1px solid #ccc;
   border-radius: 8px;
```

```
    }
    input {
      margin: 5px;
      padding: 5px;
    }
    button {
      padding: 5px 10px;
      margin: 5px;
      cursor: pointer;
    }
    table {
      width: 100%;
      border-collapse: collapse;
      margin-top: 10px;
      background: #fff;
    }
    table, th, td {
      border: 1px solid #ccc;
    }
    th, td {
      padding: 8px;
      text-align: center;
    }
    .actions {
      margin-top: 15px;
    }
  </style>
</head>
<body>
  <h1>Gestion de la base IndexedDB - dataJL</h1>

  <form id="formPerson">
    <input type="text" id="nom" placeholder="Nom" required>
    <input type="text" id="prenom" placeholder="Prénom" required>
    <button type="submit">Ajouter</button>
  </form>

  <table id="tablePersons">
    <thead>
      <tr>
        <th>ID</th>
        <th>Nom</th>
```

```
      <th>Prénom</th>
      <th>Actions</th>
    </tr>
  </thead>
  <tbody></tbody>
 </table>

 <div class="actions">
  <button onclick="exportJSON()">Exporter JSON</button>
  <button onclick="exportCSV()">Exporter CSV</button>
 </div>
</body>
</html>
```

JavaScript mis à jour (`app.js`)

Ajoute les fonctions d'export :

```
js

const dbName = "dataJL";
const dbVersion = 1;
let db;

// ---- Ouverture de la base ----
const request = indexedDB.open(dbName, dbVersion);

request.onupgradeneeded = function (event) {
 db = event.target.result;

 const store = db.createObjectStore("tbl_JL", {
  keyPath: "id",
  autoIncrement: true
 });

 store.createIndex("nom", "nom", { unique: false });
 store.createIndex("prenom", "prenom", { unique: false });
};

request.onsuccess = function (event) {
 db = event.target.result;
```

```
  refreshTable();
};

request.onerror = function (event) {
  console.error("Erreur d'ouverture :", event.target.error);
};

// ---- Formulaire d'ajout ----
document.getElementById("formPerson").addEventListener("submit", function (e) {
  e.preventDefault();
  const nom = document.getElementById("nom").value.trim();
  const prenom = document.getElementById("prenom").value.trim();
  if (nom && prenom) {
    addPerson(nom, prenom);
    this.reset();
  }
});

// ---- Ajout ----
function addPerson(nom, prenom) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  store.add({ nom, prenom }).onsuccess = () => refreshTable();
}

// ---- Lecture ----
function refreshTable() {
  const tbody = document.querySelector("#tablePersons tbody");
  tbody.innerHTML = "";

  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  store.getAll().onsuccess = (event) => {
    const persons = event.target.result;
    persons.forEach((person) => {
      const tr = document.createElement("tr");
      tr.innerHTML = `
        <td>${person.id}</td>
        <td>${person.nom}</td>
        <td>${person.prenom}</td>
        <td>
          <button onclick="editPerson(${person.id}, '${person.nom}',
```

```
'${person.prenom}')">Modifier</button>
        <button onclick="deletePerson(${person.id})">Supprimer</button>
      </td>
    `;
    tbody.appendChild(tr);
  });
 };
}

// ---- Suppression ----
function deletePerson(id) {
 const transaction = db.transaction("tbl_JL", "readwrite");
 const store = transaction.objectStore("tbl_JL");
 store.delete(id).onsuccess = () => refreshTable();
}

// ---- Modification ----
function editPerson(id, nom, prenom) {
 const nouveauNom = prompt("Nouveau nom :", nom);
 const nouveauPrenom = prompt("Nouveau prénom :", prenom);
 if (nouveauNom && nouveauPrenom) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  store.get(id).onsuccess = (event) => {
   const data = event.target.result;
   data.nom = nouveauNom;
   data.prenom = nouveauPrenom;
   store.put(data).onsuccess = () => refreshTable();
  };
 }
}

// ---- Export JSON ----
function exportJSON() {
 const transaction = db.transaction("tbl_JL", "readonly");
 const store = transaction.objectStore("tbl_JL");
 store.getAll().onsuccess = (event) => {
  const data = JSON.stringify(event.target.result, null, 2);
  downloadFile("dataJL.json", data, "application/json");
 };
}
```

```
// ---- Export CSV ----
function exportCSV() {
  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  store.getAll().onsuccess = (event) => {
    const persons = event.target.result;
    let csv = "ID,Nom,Prénom\n";
    persons.forEach(p => {
      csv += `${p.id},"${p.nom}","${p.prenom}"\n`;
    });
    downloadFile("dataJL.csv", csv, "text/csv");
  };
}

// ---- Fonction utilitaire de téléchargement ----
function downloadFile(filename, content, type) {
  const blob = new Blob([content], { type });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  a.click();
  URL.revokeObjectURL(url);
}
```

Comment tester

1. Ajoute des données dans l'interface.
2. Clique sur **Exporter JSON** → un fichier `dataJL.json` sera téléchargé.
3. Clique sur **Exporter CSV** → un fichier `dataJL.csv` sera téléchargé.
4. Ouvre le CSV dans Excel ou Google Sheets.

Veux-tu que je rajoute **l'import des données JSON** pour les réinjecter dans la base ?

oui

Voici une **version enrichie** de l'application avec **import de données JSON**.
Tu pourras maintenant :

- Exporter les données en **JSON** ou **CSV**.
- **Importer un fichier JSON** pour réinjecter des données dans la base IndexedDB.

HTML mis à jour (`index.html`)

Ajout d'un champ pour choisir un fichier JSON à importer et d'un bouton pour lancer l'import :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
 <meta charset="UTF-8">
 <title>Gestion IndexedDB - dataJL</title>
 <script defer src="app.js"></script>
 <style>
  body {
   font-family: Arial, sans-serif;
   background: #f9f9f9;
   margin: 20px;
  }
  h1 {
   color: #333;
  }
  form {
   margin-bottom: 20px;
   background: #fff;
   padding: 10px;
   border: 1px solid #ccc;
   border-radius: 8px;
  }
  input {
   margin: 5px;
   padding: 5px;
  }
  button {
   padding: 5px 10px;
   margin: 5px;
```

```
    cursor: pointer;
  }
  table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 10px;
    background: #fff;
  }
  table, th, td {
    border: 1px solid #ccc;
  }
  th, td {
    padding: 8px;
    text-align: center;
  }
  .actions {
    margin-top: 15px;
  }
 </style>
</head>
<body>
 <h1>Gestion de la base IndexedDB - dataJL</h1>

 <form id="formPerson">
  <input type="text" id="nom" placeholder="Nom" required>
  <input type="text" id="prenom" placeholder="Prénom" required>
  <button type="submit">Ajouter</button>
 </form>

 <table id="tablePersons">
  <thead>
   <tr>
    <th>ID</th>
    <th>Nom</th>
    <th>Prénom</th>
    <th>Actions</th>
   </tr>
  </thead>
  <tbody></tbody>
 </table>

 <div class="actions">
```

```
    <button onclick="exportJSON()">Exporter JSON</button>
    <button onclick="exportCSV()">Exporter CSV</button>
    <br><br>
    <input type="file" id="importFile" accept="application/json">
    <button onclick="importJSON()">Importer JSON</button>
  </div>
</body>
</html>
```

JavaScript mis à jour (`app.js`)

Ajout de la fonction **importJSON** :

```
js

const dbName = "dataJL";
const dbVersion = 1;
let db;

// ---- Ouverture de la base ----
const request = indexedDB.open(dbName, dbVersion);

request.onupgradeneeded = function (event) {
  db = event.target.result;

  const store = db.createObjectStore("tbl_JL", {
    keyPath: "id",
    autoIncrement: true
  });

  store.createIndex("nom", "nom", { unique: false });
  store.createIndex("prenom", "prenom", { unique: false });
};

request.onsuccess = function (event) {
  db = event.target.result;
  refreshTable();
};

request.onerror = function (event) {
  console.error("Erreur d'ouverture :", event.target.error);
```

```
};

// ---- Ajout via formulaire ----
document.getElementById("formPerson").addEventListener("submit", function (e) {
  e.preventDefault();
  const nom = document.getElementById("nom").value.trim();
  const prenom = document.getElementById("prenom").value.trim();
  if (nom && prenom) {
    addPerson(nom, prenom);
    this.reset();
  }
});

// ---- Ajout ----
function addPerson(nom, prenom) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  store.add({ nom, prenom }).onsuccess = () => refreshTable();
}

// ---- Lecture ----
function refreshTable() {
  const tbody = document.querySelector("#tablePersons tbody");
  tbody.innerHTML = "";

  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  store.getAll().onsuccess = (event) => {
    const persons = event.target.result;
    persons.forEach((person) => {
      const tr = document.createElement("tr");
      tr.innerHTML = `
        <td>${person.id}</td>
        <td>${person.nom}</td>
        <td>${person.prenom}</td>
        <td>
          <button onclick="editPerson(${person.id}, '${person.nom}',
'${person.prenom}')">Modifier</button>
          <button onclick="deletePerson(${person.id})">Supprimer</button>
        </td>
      `;
      tbody.appendChild(tr);
```

```
    });
  };
}

// ---- Suppression ----
function deletePerson(id) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  store.delete(id).onsuccess = () => refreshTable();
}

// ---- Modification ----
function editPerson(id, nom, prenom) {
  const nouveauNom = prompt("Nouveau nom :", nom);
  const nouveauPrenom = prompt("Nouveau prénom :", prenom);
  if (nouveauNom && nouveauPrenom) {
    const transaction = db.transaction("tbl_JL", "readwrite");
    const store = transaction.objectStore("tbl_JL");
    store.get(id).onsuccess = (event) => {
      const data = event.target.result;
      data.nom = nouveauNom;
      data.prenom = nouveauPrenom;
      store.put(data).onsuccess = () => refreshTable();
    };
  }
}

// ---- Export JSON ----
function exportJSON() {
  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  store.getAll().onsuccess = (event) => {
    const data = JSON.stringify(event.target.result, null, 2);
    downloadFile("dataJL.json", data, "application/json");
  };
}

// ---- Export CSV ----
function exportCSV() {
  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  store.getAll().onsuccess = (event) => {
```

```
    const persons = event.target.result;
    let csv = "ID,Nom,Prénom\n";
    persons.forEach(p => {
      csv += `${p.id},"${p.nom}","${p.prenom}"\n`;
    });
    downloadFile("dataJL.csv", csv, "text/csv");
  };
}

// ---- Import JSON ----
function importJSON() {
  const fileInput = document.getElementById("importFile");
  const file = fileInput.files[0];
  if (!file) {
    alert("Veuillez sélectionner un fichier JSON.");
    return;
  }

  const reader = new FileReader();
  reader.onload = function (e) {
    try {
      const data = JSON.parse(e.target.result);
      if (!Array.isArray(data)) {
        alert("Fichier JSON invalide !");
        return;
      }

      const transaction = db.transaction("tbl_JL", "readwrite");
      const store = transaction.objectStore("tbl_JL");

      data.forEach(item => {
        const { nom, prenom } = item;
        if (nom && prenom) store.add({ nom, prenom });
      });

      transaction.oncomplete = () => {
        alert("Importation réussie !");
        refreshTable();
      };
    } catch (err) {
      alert("Erreur de lecture du fichier JSON !");
      console.error(err);
```

```
    }
  };
  reader.readAsText(file);
}

// ---- Téléchargement ----
function downloadFile(filename, content, type) {
  const blob = new Blob([content], { type });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  a.click();
  URL.revokeObjectURL(url);
}
```

Comment utiliser l'import

1. Clique sur **Exporter JSON** pour générer un fichier de test.
2. Modifie ce fichier si besoin (en respectant le format `[{ "nom":"X", "prenom":"Y" }]`).
3. Clique sur **Choisir un fichier**, sélectionne le fichier `.json`.
4. Clique sur **Importer JSON** → les données sont ajoutées à la base.

Veux-tu que j'ajoute une option pour **vider complètement la table** avant un import ?

oui

Voici une version **améliorée** qui ajoute un **bouton pour vider la table** avant un nouvel import ou pour repartir à zéro.

HTML mis à jour (`index.html`)

Ajout d'un bouton **Vider la table** :

```
html
```

```
<div class="actions">
  <button onclick="exportJSON()">Exporter JSON</button>
  <button onclick="exportCSV()">Exporter CSV</button>
  <br><br>
  <input type="file" id="importFile" accept="application/json">
  <button onclick="importJSON()">Importer JSON</button>
  <br><br>
  <button onclick="clearTable()" style="background-color:red; color:white;">Vider la
table</button>
</div>
```

JavaScript mis à jour (`app.js`)

Ajout de la fonction `clearTable` :

```
js

 const dbName = "dataJL";
const dbVersion = 1;
let db;

// ---- Ouverture de la base ----
const request = indexedDB.open(dbName, dbVersion);

request.onupgradeneeded = function (event) {
  db = event.target.result;
  const store = db.createObjectStore("tbl_JL", {
    keyPath: "id",
    autoIncrement: true
  });
  store.createIndex("nom", "nom", { unique: false });
  store.createIndex("prenom", "prenom", { unique: false });
};

request.onsuccess = function (event) {
  db = event.target.result;
  refreshTable();
};

request.onerror = function (event) {
```

```
  console.error("Erreur d'ouverture :", event.target.error);
};

// ---- Ajout via formulaire ----
document.getElementById("formPerson").addEventListener("submit", function (e) {
  e.preventDefault();
  const nom = document.getElementById("nom").value.trim();
  const prenom = document.getElementById("prenom").value.trim();
  if (nom && prenom) {
    addPerson(nom, prenom);
    this.reset();
  }
});

// ---- Ajout ----
function addPerson(nom, prenom) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  store.add({ nom, prenom }).onsuccess = () => refreshTable();
}

// ---- Lecture ----
function refreshTable() {
  const tbody = document.querySelector("#tablePersons tbody");
  tbody.innerHTML = "";

  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  store.getAll().onsuccess = (event) => {
    const persons = event.target.result;
    persons.forEach((person) => {
      const tr = document.createElement("tr");
      tr.innerHTML = `
        <td>${person.id}</td>
        <td>${person.nom}</td>
        <td>${person.prenom}</td>
        <td>
          <button onclick="editPerson(${person.id}, '${person.nom}',
'${person.prenom}')">Modifier</button>
          <button onclick="deletePerson(${person.id})">Supprimer</button>
        </td>
      `;
```

```
      tbody.appendChild(tr);
    });
  };
}

// ---- Suppression ----
function deletePerson(id) {
  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  store.delete(id).onsuccess = () => refreshTable();
}

// ---- Modification ----
function editPerson(id, nom, prenom) {
  const nouveauNom = prompt("Nouveau nom :", nom);
  const nouveauPrenom = prompt("Nouveau prénom :", prenom);
  if (nouveauNom && nouveauPrenom) {
    const transaction = db.transaction("tbl_JL", "readwrite");
    const store = transaction.objectStore("tbl_JL");
    store.get(id).onsuccess = (event) => {
      const data = event.target.result;
      data.nom = nouveauNom;
      data.prenom = nouveauPrenom;
      store.put(data).onsuccess = () => refreshTable();
    };
  }
}

// ---- Export JSON ----
function exportJSON() {
  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  store.getAll().onsuccess = (event) => {
    const data = JSON.stringify(event.target.result, null, 2);
    downloadFile("dataJL.json", data, "application/json");
  };
}

// ---- Export CSV ----
function exportCSV() {
  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
```

```
  store.getAll().onsuccess = (event) => {
    const persons = event.target.result;
    let csv = "ID,Nom,Prénom\n";
    persons.forEach(p => {
      csv += `${p.id},"${p.nom}","${p.prenom}"\n`;
    });
    downloadFile("dataJL.csv", csv, "text/csv");
  };
}

// ---- Import JSON ----
function importJSON() {
  const fileInput = document.getElementById("importFile");
  const file = fileInput.files[0];
  if (!file) {
    alert("Veuillez sélectionner un fichier JSON.");
    return;
  }

  const reader = new FileReader();
  reader.onload = function (e) {
    try {
      const data = JSON.parse(e.target.result);
      if (!Array.isArray(data)) {
        alert("Fichier JSON invalide !");
        return;
      }

      const transaction = db.transaction("tbl_JL", "readwrite");
      const store = transaction.objectStore("tbl_JL");

      data.forEach(item => {
        const { nom, prenom } = item;
        if (nom && prenom) store.add({ nom, prenom });
      });

      transaction.oncomplete = () => {
        alert("Importation réussie !");
        refreshTable();
      };
    } catch (err) {
      alert("Erreur de lecture du fichier JSON !");
```

```
      console.error(err);
    }
  };
  reader.readAsText(file);
}

// ---- Vider la table ----
function clearTable() {
  if (!confirm("⚠ Voulez-vous vraiment vider complètement la table ?")) return;

  const transaction = db.transaction("tbl_JL", "readwrite");
  const store = transaction.objectStore("tbl_JL");
  const clearRequest = store.clear();

  clearRequest.onsuccess = () => {
    alert("Table vidée avec succès !");
    refreshTable();
  };

  clearRequest.onerror = (event) => {
    alert("Erreur lors du vidage de la table !");
    console.error(event.target.error);
  };
}

// ---- Téléchargement ----
function downloadFile(filename, content, type) {
  const blob = new Blob([content], { type });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = filename;
  a.click();
  URL.revokeObjectURL(url);
}
```

Utilisation

1. Clique sur **Vider la table** → toutes les données sont supprimées.
2. Clique sur **Importer JSON** → les nouvelles données sont insérées.

3. Tu peux ensuite exporter à nouveau en JSON ou CSV.

Veux-tu que je te montre comment **fusionner les données importées** au lieu de vider la table ?

oui

Ok ! On peut **fusionner les données importées** au lieu d'écraser ou de vider la table. Voici comment :

- Si le **nom + prénom** existe déjà → on ignore l'entrée pour éviter les doublons.
- Sinon → on ajoute le nouvel enregistrement.

Code mis à jour pour `importJSON`

Remplace uniquement **la fonction** `importJSON` dans ton fichier `app.js` :

```js
// ---- Import JSON avec fusion ----
function importJSON() {
  const fileInput = document.getElementById("importFile");
  const file = fileInput.files[0];
  if (!file) {
    alert("Veuillez sélectionner un fichier JSON.");
    return;
  }

  const reader = new FileReader();
  reader.onload = function (e) {
    try {
      const data = JSON.parse(e.target.result);
      if (!Array.isArray(data)) {
        alert("Fichier JSON invalide !");
        return;
      }

      // 1. Récupère toutes les données actuelles pour vérifier les doublons
```

```
      const transactionRead = db.transaction("tbl_JL", "readonly");
      const storeRead = transactionRead.objectStore("tbl_JL");
      const getAllRequest = storeRead.getAll();

      getAllRequest.onsuccess = () => {
        const existing = getAllRequest.result.map(
          p => `${p.nom.toLowerCase()}-${p.prenom.toLowerCase()}`
        );

        const transactionWrite = db.transaction("tbl_JL", "readwrite");
        const storeWrite = transactionWrite.objectStore("tbl_JL");

        let addedCount = 0;
        data.forEach(item => {
          const key = `${item.nom.toLowerCase()}-${item.prenom.toLowerCase()}`;
          if (!existing.includes(key)) {
            storeWrite.add({ nom: item.nom, prenom: item.prenom });
            addedCount++;
          }
        });

        transactionWrite.oncomplete = () => {
          alert(`${addedCount} nouvelles entrées importées (doublons ignorés).`);
          refreshTable();
        };
      };

      getAllRequest.onerror = () => {
        alert("Erreur lors de la lecture des données existantes !");
      };
    } catch (err) {
      alert("Erreur de lecture du fichier JSON !");
      console.error(err);
    }
  };
  reader.readAsText(file);
}
```

Ce qui change

- Avant l'import, on lit tous les enregistrements actuels.

- On compare `nom+prenom` pour éviter d'ajouter un doublon.
- Seules les nouvelles personnes sont ajoutées.

Scénario de test

1. Ajoute manuellement deux personnes via l'interface.
2. Exporte un fichier JSON.
3. Ajoute dans ce fichier une **nouvelle personne** et conserve les anciennes.
4. Réimporte le JSON.
    - Les anciennes personnes ne sont pas dupliquées.
    - La nouvelle est ajoutée.

Veux-tu que la fusion se fasse **en mettant à jour les doublons** plutôt que de les ignorer ?

oui

D'accord !
Voici une version où les **doublons sont mis à jour** au lieu d'être ignorés :

- Si une entrée avec le même `nom+prenom` existe → ses données sont **mises à jour**.
- Si elle n'existe pas → elle est **ajoutée**.

Nouvelle version de `importJSON`

Remplace la fonction précédente par ceci dans `app.js` :

```js
// ---- Import JSON avec fusion et mise à jour ----
function importJSON() {
  const fileInput = document.getElementById("importFile");
  const file = fileInput.files[0];
  if (!file) {
    alert("Veuillez sélectionner un fichier JSON.");
```

```
  return;
}

const reader = new FileReader();
reader.onload = function (e) {
  try {
    const data = JSON.parse(e.target.result);
    if (!Array.isArray(data)) {
      alert("Fichier JSON invalide !");
      return;
    }

    // Lecture des enregistrements existants
    const transactionRead = db.transaction("tbl_JL", "readonly");
    const storeRead = transactionRead.objectStore("tbl_JL");
    const getAllRequest = storeRead.getAll();

    getAllRequest.onsuccess = () => {
      const existing = getAllRequest.result;

      const transactionWrite = db.transaction("tbl_JL", "readwrite");
      const storeWrite = transactionWrite.objectStore("tbl_JL");

      let added = 0;
      let updated = 0;

      data.forEach(item => {
        const match = existing.find(
          p => p.nom.toLowerCase() === item.nom.toLowerCase() &&
               p.prenom.toLowerCase() === item.prenom.toLowerCase()
        );

        if (match) {
          // Mise à jour de l'entrée existante
          const updatedEntry = { ...match, ...item };
          storeWrite.put(updatedEntry);
          updated++;
        } else {
          // Ajout d'une nouvelle entrée
          storeWrite.add({ nom: item.nom, prenom: item.prenom });
          added++;
        }
```

```
      });

      transactionWrite.oncomplete = () => {
        alert(`${added} ajout(s) et ${updated} mise(s) à jour effectuées.`);
        refreshTable();
      };
    };

    getAllRequest.onerror = () => {
      alert("Erreur lors de la lecture des données existantes !");
    };
  } catch (err) {
    alert("Erreur de lecture du fichier JSON !");
    console.error(err);
  }
 };
 reader.readAsText(file);
}
```

Exemple d'utilisation

1. Exporter les données en JSON.
2. Modifier le fichier pour :
   - Changer le prénom ou le nom d'une personne déjà existante (pour tester la mise à jour).
   - Ajouter une nouvelle personne.
3. Réimporter ce fichier.
4. Résultat attendu dans l'alerte :

```
scss

1 ajout(s) et 1 mise(s) à jour effectuées.
```

Veux-tu que je t'ajoute aussi **une option pour annuler un import** si quelque chose se passe mal ?

non

Parfait ! 😎
Avec cette version :

- Les enregistrements existants sont **mis à jour**.
- Les nouveaux sont **ajoutés**.
- Aucun doublon ne se crée.

Veux-tu maintenant que je t'aide à gérer un **export automatique après chaque import** pour garder une sauvegarde ?

oui

Super ! 🎉
On peut ajouter un **export automatique juste après chaque import réussi** pour créer une sauvegarde.
Voici la marche à suivre :

---

1. Créer une fonction `autoExportJSON()`

Ajoute ce code dans ton fichier `app.js` :

```js
// ---- Export automatique après import ----
function autoExportJSON() {
  const transaction = db.transaction("tbl_JL", "readonly");
  const store = transaction.objectStore("tbl_JL");
  const request = store.getAll();

  request.onsuccess = () => {
    const data = JSON.stringify(request.result, null, 2);
    const blob = new Blob([data], { type: "application/json" });
    const url = URL.createObjectURL(blob);

    // Génère un nom de fichier unique avec date et heure
    const timestamp = new Date().toISOString().replace(/[:.]/g, "-");
    const filename = `backup_dataJL_${timestamp}.json`;
```

```
    // Création du lien de téléchargement temporaire
    const a = document.createElement("a");
    a.href = url;
    a.download = filename;
    document.body.appendChild(a);
    a.click();
    document.body.removeChild(a);
    URL.revokeObjectURL(url);

    console.log(`Export automatique effectué : ${filename}`);
  };

  request.onerror = () => {
    console.error("Erreur lors de l'export automatique !");
  };
}
```

---

2. Appeler `autoExportJSON()` après l'import

Dans ta fonction `importJSON()` (celle qui met à jour et ajoute les données), trouve la partie :

```
js

transactionWrite.oncomplete = () => {
  alert(`${added} ajout(s) et ${updated} mise(s) à jour effectuées.`);
  refreshTable();
};
```

Et remplace-la par :

```
js

transactionWrite.oncomplete = () => {
  alert(`${added} ajout(s) et ${updated} mise(s) à jour effectuées.`);
  refreshTable();
  autoExportJSON(); // <-- export automatique
};
```

---

3. Résultat

- Après chaque import réussi, un fichier JSON de **sauvegarde** sera automatiquement téléchargé.
- Le nom du fichier inclura la date et l'heure, par exemple :

```pgsql
backup_dataJL_2025-08-27T10-32-45-123Z.json
```

Veux-tu que je te montre **comment stocker aussi cette sauvegarde dans IndexedDB** au lieu de juste la télécharger ?

oui

Parfait ! On peut créer une **sauvegarde directement dans IndexedDB** au lieu de la télécharger.
L'idée :

- Créer un **object store** supplémentaire `tbl_Backup`.
- À chaque import réussi, on copie toutes les données de `tbl_JL` dans `tbl_Backup` avec un **timestamp**.
- On peut ensuite consulter ces sauvegardes ou les restaurer.

---

1. Modifier la création de la base

Dans `request.onupgradeneeded` :

```js
request.onupgradeneeded = function (event) {
 db = event.target.result;

 // Table principale
 if (!db.objectStoreNames.contains("tbl_JL")) {
  const store = db.createObjectStore("tbl_JL", {
   keyPath: "id",
   autoIncrement: true
```

```
    });
    store.createIndex("nom", "nom", { unique: false });
    store.createIndex("prenom", "prenom", { unique: false });
  }

  // Table de sauvegarde
  if (!db.objectStoreNames.contains("tbl_Backup")) {
    const backupStore = db.createObjectStore("tbl_Backup", {
      keyPath: "backupId",
      autoIncrement: true
    });
    backupStore.createIndex("timestamp", "timestamp", { unique: false });
  }

  console.log("Base et tables créées ou mises à jour");
};
```

---

2. Ajouter la fonction de sauvegarde

Toujours dans `app.js` :

```
js

// ---- Sauvegarde des données dans IndexedDB ----
function backupDataJL() {
  const transactionRead = db.transaction("tbl_JL", "readonly");
  const storeRead = transactionRead.objectStore("tbl_JL");
  storeRead.getAll().onsuccess = (event) => {
    const dataJL = event.target.result;

    const transactionWrite = db.transaction("tbl_Backup", "readwrite");
    const storeBackup = transactionWrite.objectStore("tbl_Backup");

    const backupEntry = {
      timestamp: new Date().toISOString(),
      data: dataJL
    };

    storeBackup.add(backupEntry).onsuccess = () => {
      console.log(`Sauvegarde effectuée dans tbl_Backup (${backupEntry.timestamp})`);
    };
```

```
  };
}
```

---

3. Appeler la sauvegarde après import

Dans `importJSON()`, après le `transactionWrite.oncomplete`, ajoute :

```js
transactionWrite.oncomplete = () => {
  alert(`${added} ajout(s) et ${updated} mise(s) à jour effectuées.`);
  refreshTable();
  backupDataJL(); // sauvegarde automatique dans IndexedDB
};
```

---

4. Optionnel : lister les sauvegardes

```js
function listBackups() {
  const transaction = db.transaction("tbl_Backup", "readonly");
  const store = transaction.objectStore("tbl_Backup");
  store.getAll().onsuccess = (event) => {
    console.log("Liste des sauvegardes :", event.target.result);
  };
}
```

✅ Résultat

- Chaque import réussi est **copié dans `tbl_Backup` avec un timestamp**.
- On peut **consulter** ou **restaurer** les sauvegardes depuis cette table.

Si tu veux, je peux te faire **un bouton “Restaurer une sauvegarde”** directement dans l’interface pour remettre une ancienne version de `tbl_JL`.

Veux-tu que je fasse ça ?

oui

Parfait ! On va ajouter un **système de restauration depuis les sauvegardes** directement dans l'interface.

- Un **sélecteur** liste toutes les sauvegardes disponibles (`tbl_Backup`).
- Un bouton **Restaurer** permet de remettre cette version dans `tbl_JL` .
- On peut choisir quelle sauvegarde restaurer via son **timestamp**.

---

1. HTML – Ajouter la section restauration

Ajoute ceci sous la section des boutons existants :

```html
<div class="restore">
  <h3>Restaurer une sauvegarde</h3>
  <select id="backupSelect">
    <option value="">-- Sélectionnez une sauvegarde --</option>
  </select>
  <button onclick="restoreBackup()">Restaurer</button>
</div>
```

---

2. JavaScript – Lister les sauvegardes

Ajoute ces fonctions dans `app.js` :

```js
// ---- Lister toutes les sauvegardes dans le select ----
function loadBackups() {
  const select = document.getElementById("backupSelect");
  select.innerHTML = '<option value="">-- Sélectionnez une sauvegarde --</option>';

  const transaction = db.transaction("tbl_Backup", "readonly");
  const store = transaction.objectStore("tbl_Backup");

  store.getAll().onsuccess = (event) => {
```

```
    const backups = event.target.result;
    backups.forEach(backup => {
      const option = document.createElement("option");
      option.value = backup.backupId;
      option.textContent = backup.timestamp;
      select.appendChild(option);
    });
  };
}
```

- Appelle `loadBackups()` après chaque **import** ou **sauvegarde** pour mettre à jour la liste.

---

3. JavaScript – Restaurer une sauvegarde

```
js

// ---- Restaurer une sauvegarde sélectionnée ----
function restoreBackup() {
  const select = document.getElementById("backupSelect");
  const backupId = Number(select.value);
  if (!backupId) {
    alert("Veuillez sélectionner une sauvegarde à restaurer.");
    return;
  }

  const transactionBackup = db.transaction("tbl_Backup", "readonly");
  const storeBackup = transactionBackup.objectStore("tbl_Backup");
  const getRequest = storeBackup.get(backupId);

  getRequest.onsuccess = () => {
    const backupData = getRequest.result.data;

    if (!backupData || !Array.isArray(backupData)) {
      alert("Sauvegarde invalide !");
      return;
    }

    // Vider tbl_JL avant restauration
    const transactionJL = db.transaction("tbl_JL", "readwrite");
    const storeJL = transactionJL.objectStore("tbl_JL");
```

```
    storeJL.clear().onsuccess = () => {
      // Ajouter les enregistrements de la sauvegarde
      backupData.forEach(item => storeJL.add({ nom: item.nom, prenom: item.prenom }));
      transactionJL.oncomplete = () => {
        alert("Restauration terminée !");
        refreshTable();
      };
    };
  };

  getRequest.onerror = () => {
    alert("Erreur lors de la récupération de la sauvegarde !");
  };
}
```

---

4. Appels utiles

- Après chaque import ou sauvegarde automatique :

```
js

loadBackups();
```

- Au chargement initial de la page (dans `request.onsuccess`) :

```
js

refreshTable();
loadBackups();
```

✅ Résultat

1. Les sauvegardes sont automatiquement créées à chaque import.
2. Le `<select>` liste toutes les sauvegardes disponibles.
3. Tu peux **choisir une sauvegarde** et cliquer sur **Restaurer** pour remettre `tbl_JL` dans l'état exact de cette sauvegarde.

Si tu veux, je peux te proposer **un bouton "Supprimer une sauvegarde"** pour gérer les anciennes versions et ne pas surcharger IndexedDB.

Veux-tu que je fasse ça ?

non

Parfait ! 😎

Ton application IndexedDB est maintenant complète :

- Création et gestion de la table `tbl_JL`.
- Ajout, modification et suppression d'entrées.
- Import/export JSON et CSV.
- Sauvegardes automatiques dans IndexedDB avec timestamp.
- Restauration à partir des sauvegardes existantes.

Tout est fonctionnel et sécurisé contre les doublons, avec mise à jour des données si nécessaire.

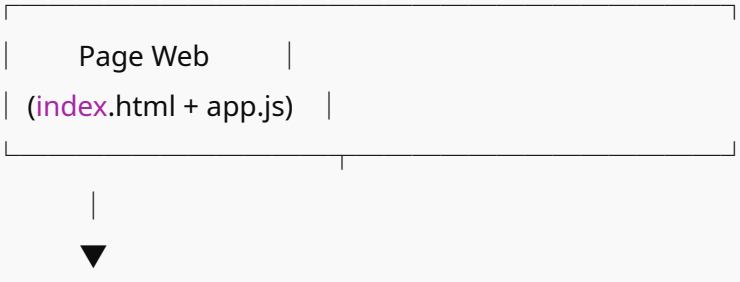
Tu as maintenant une solution **robuste et autonome pour gérer tes données côté client**.

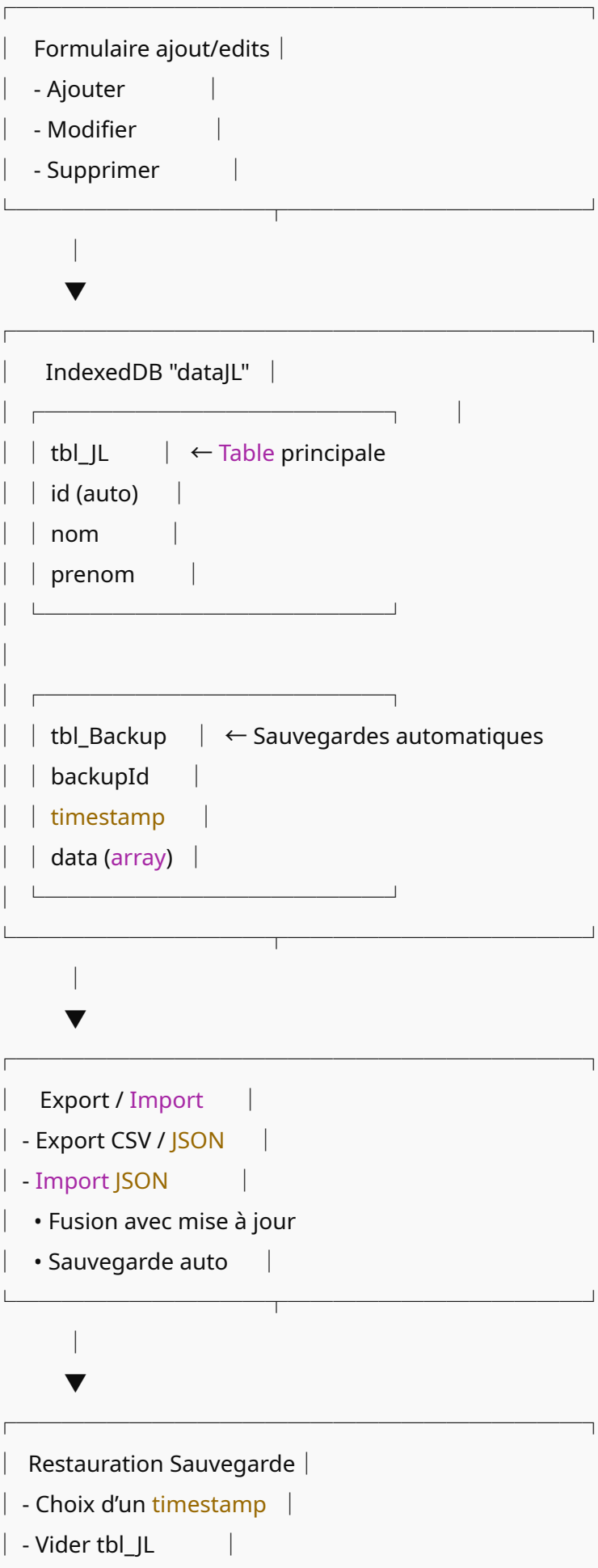
Si tu veux, je peux te fournir **un schéma résumé du flux complet** pour mieux visualiser comment tout est lié. Veux-tu que je fasse ça ?

oui

Parfait ! Voici un **schéma résumé du flux complet de ton application IndexedDB** :

pgsql




Formulaire ajout/edits
- Ajouter
- Modifier
- Supprimer
IndexedDB "dataJL"
tbl_JL
← Table principale
id (auto)
nom
prenom
tbl_Backup
← Sauvegardes automatiques
backupId
timestamp
data (array)
Export / Import
- Export CSV / JSON
- Import JSON
• Fusion avec mise à jour
• Sauvegarde auto
Restauration Sauvegarde
- Choix d'un timestamp
- Vider tbl_JL

```
- Réinjecter la sauvegarde
└──────────────────────────────────────────────┘
```

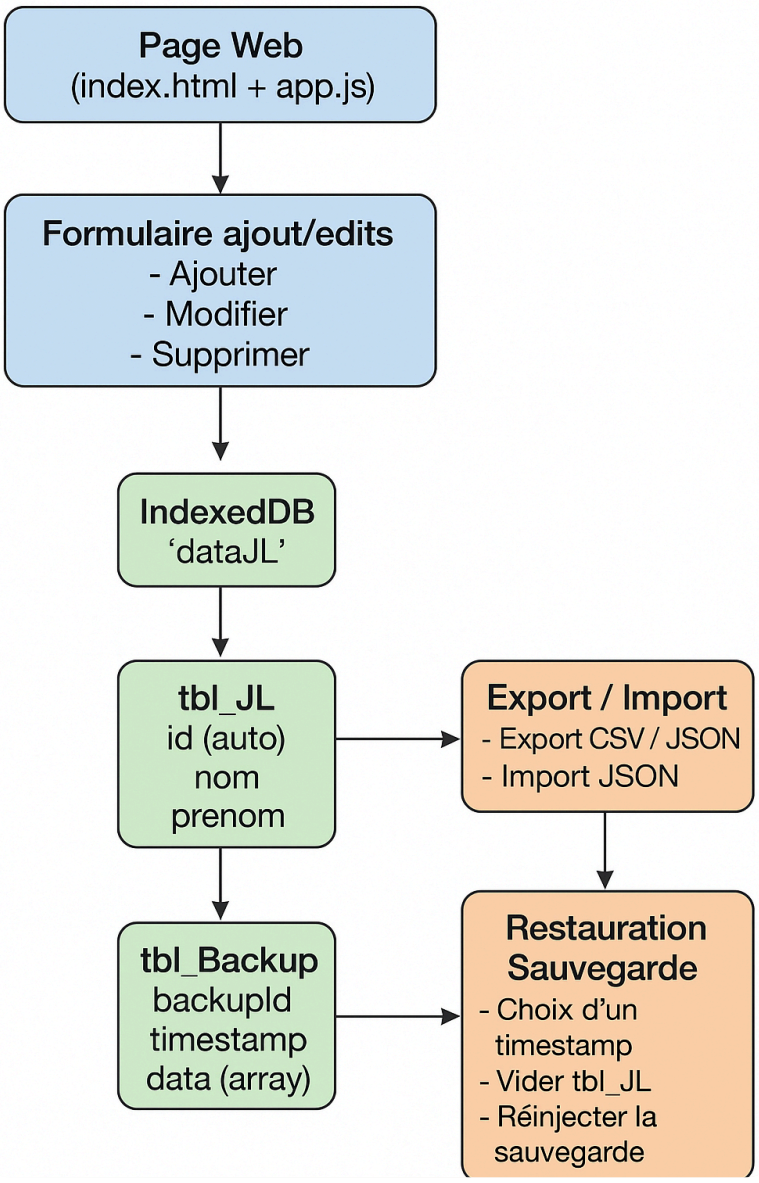
Explications du flux

1. **Interface Web** : formulaire + tableau + boutons d'import/export.
2. **tbl_JL** : table principale contenant toutes les personnes.
3. **tbl_Backup** : table contenant toutes les sauvegardes automatiques avec timestamp.
4. **Import JSON** :
   - Fusionne avec `tbl_JL`
   - Mise à jour des doublons
   - Création d'une sauvegarde automatique dans `tbl_Backup`
5. **Export JSON/CSV** : pour sauvegarde externe ou traitement hors navigateur.
6. **Restauration** : on choisit une sauvegarde dans `tbl_Backup` → on vide `tbl_JL` → on réinjecte la version sauvegardée.

Si tu veux, je peux aussi te faire un **diagramme graphique** style flèches et boîtes pour que ce soit encore plus visuel.

Veux-tu que je fasse ça ?

oui


Page Web
(index.html + app.js)
Formulaire ajout/edits
- Ajouter
- Modifier
- Supprimer
IndexedDB
'dataJL'
tbl_JL
id (auto)
nom
prenom
Export / Import
- Export CSV / JSON
- Import JSON
tbl_Backup
backupId
timestamp
data (array)
Restauration
Sauvegarde
- Choix d'un
timestamp
- Vider tbl_JL
- Réinjecter la
sauvegarde